



Secure Cloud-Based Banking System Deployment Using AWS VPC Architecture

A defense-in-depth banking web platform built on Amazon Web Services — network-segmented, encrypted end-to-end, and load-tested for financial-grade reliability.

UPTIME

99.8%

AVG. RESPONSE

1.32_s

PEAK THROUGHPUT

140_{tx/s}

ENCRYPTION

AES-256

OVERVIEW

The problem this solves

Financial institutions moving to the cloud face a narrow path: scale to meet demand while satisfying PCI-DSS and GDPR-grade security expectations. This project designs, deploys, and load-tests a three-tier banking web platform entirely within an **AWS Virtual Private Cloud** — proving that cloud-native infrastructure can match or exceed on-premise security while cutting cost and improving resilience.

ARCHITECTURE

Three-tier system design

01 Presentation Layer

HTML5 / CSS3 / PHP front end — login, balance checks, transfers, password management, loan requests.

02 Application Layer

PHP business logic on AWS EC2 (Amazon Linux 2 + Apache), deployed across public/private subnets, talking to the database only over secured internal routes.

03 Database Layer

AWS RDS (MySQL) in a private subnet — no public access. All data encrypted at rest and in transit via AWS KMS.

04 Security Infrastructure

IAM least-privilege roles, Network ACLs, Security Groups, TLS 1.2 everywhere, and CloudTrail + GuardDuty for continuous monitoring.

Deployment sequence

STEP	ACTION
1. VPC Design	Custom VPC (CIDR 10.0.0.0/16) with public/private subnets across multiple AZs for fault tolerance
2. EC2 Configuration	Amazon Linux 2 in public subnet — Apache + PHP, hardened firewall rules
3. RDS Setup	MySQL RDS in private subnet, reachable only from application-tier EC2 instances
4. IAM Framework	Separate roles for admins, DB users, and security auditors — least-privilege
5. App Deployment	Code shipped to EC2 via SSH with secured, environment-specific DB config
6. Monitoring	CloudWatch for live metrics; CloudTrail for audit logs and anomaly alerts

PERFORMANCE

Load-test results

1.32s	175ms	140/s	99.8%
RESPONSE TIME @ 200 USERS	DB QUERY LATENCY UNDER LOAD	PEAK TRANSACTION THROUGHPUT	UPTIME, 1-MONTH WINDOW

Tested with Apache JMeter and AWS Performance Insights. Versus an equivalent on-premise setup, the AWS deployment showed a **38% faster response time** and a **26% higher throughput**, attributable to elastic resource allocation.

SECURITY

Defense-in-depth controls

Network Isolation	TLS 1.2 in Transit	AES-256 at Rest	IAM Least-Privilege	CloudTrail Audit Logs	GuardDuty Threat Detection
-------------------	--------------------	-----------------	---------------------	-----------------------	----------------------------

The database tier has no public internet path — reachable only via tightly-scoped Network ACLs and Security Groups from the application layer. This layered approach directly mitigates common web vulnerabilities: SQL injection, unauthorized breach, script injection, and credential theft.

BENCHMARK

Cloud vs. on-premise

Conventional On-Premise	Proposed AWS System
Hardware-dependent deployment	Cloud-based (EC2, RDS, VPC)
Limited vertical scaling	Auto-scaling, elastic load balancing
Basic firewall protection	IAM + TLS + VPC isolation
Manual updates and patches	Automated patching via AWS
High fixed capital expenditure	Pay-as-you-go operational model
Local storage, limited encryption	AES-256 at rest & in transit
Complex, expensive disaster recovery	Built-in multi-AZ redundancy
Manual audit processes	Automated compliance reporting

Future work

- **Multi-Factor Authentication** — bio-verification and hardware security keys beyond passwords
- **AI-based fraud detection** — ML models flagging abnormal transaction patterns pre-emptively
- **Blockchain integration** — immutable transaction ledgers for reconciliation transparency
- **Containerization** — Docker + AWS ECS for scalable, auto-recovering deployment
- **Advanced analytics** — QuickSight pipelines for transaction and usage intelligence